

Reviewer Pack — Minimal Rank of Quadratic Forms vs Communication Complexity ...

Entry e4f63c2745bc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 22:56:09 UTC

Minimal Rank of Quadratic Forms vs Communication Complexity for Disjointness

Entry ID: e4f63c2745bc **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 22:56:09 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quadratic Forms) **Field B** (complexity object): Communication Complexity (Disjointness)

Statement:

{'text': 'For every $N \times N$ Boolean matrix M with $N \leq 40$, the minimal rank of its associated quadratic form is monotonically related to the randomized communication complexity for the disjointness problem with input M . Specifically, if M has a low rank, then the disjointness problem with input M has a high communication complexity.', 'quantitative': 'E[min_rank(M)] = Θ (communication_complexity_disjointness(M))'}

Rationale (proposer's reasoning):

{'text': 'Quadratic forms provide a generalization of polynomials that can encode logical constraints and relations. By examining the minimal rank of quadratic forms associated with Boolean matrices, we may expose hidden structures related to communication complexity, potentially revealing new insights into the hardness of disjointness problems.', 'quantitative': 'The use of quadratic forms in encoding logical constraints is a less explored area in complexity theory, suggesting that they might uncover novel relationships between geometric and logical properties.'}

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d1a2d1bba7c2bf08

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for at least 24 out of 30 random seeds, the correlation coefficient between min_rank(Q(M)) and communication_complexity_disjointness(M) exceeds 0.8 AND the mean difference between min_rank(Q(M)) and communication_complexity_disjointness(M) is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=2.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_matrix(n):
        return [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    def matrix_multiplication(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i+1, n):
                factor = A[j][i] / A[i][i]
                for k in range(i, n):
                    A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]
        x = [0] * n
```

```

    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def min_rank(matrix):
    n = len(matrix)
    rank = 0
    augmented_matrix = [row + [0] for row in matrix]
    for i in range(n):
        if augmented_matrix[i][i] == 0:
            for j in range(i+1, n):
                if augmented_matrix[j][i] != 0:
                    augmented_matrix[i], augmented_matrix[j] = augmented_matrix[j], augmented_matrix[i]
                    break
            else:
                continue
        rank += 1
        for j in range(n):
            if j == i:
                continue
            factor = augmented_matrix[j][i] / augmented_matrix[i][i]
            for k in range(n+1):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return rank

def communication_complexity_disjointness(M):
    n = len(M)
    count = 0
    for i in range(n):
        for j in range(i+1, n):
            if M[i][j] == 1:
                count += 1
    return count

n = random.randint(5, 40)
M = generate_boolean_matrix(n)
Q_M = matrix_multiplication(M, M)
rank = min_rank(Q_M)
comm_complexity = communication_complexity_disjointness(M)

return {
    "metric_name": "communication_complexity",
    "metric_value": comm_complexity,
    "instances_tested": 1,
    "conjecture_holds": rank <= comm_complexity,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}
elif any(not r["conjecture_holds"] for r in results):
    counterexample = next(r for r in results if not r["conjecture_holds"])["counterexample"]
    RESULT: FALSIFIED counterexample="{counterexample}" first_failing_seed={seeds[next(i for i for i in seeds if not r["conjecture_holds"])]}
else:
    RESULT: INCONCLUSIVE reason=insufficient_evidence

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}
TRIAL: {'metric_name': 'min_rank(Q(M))', 'metric_value': 40, 'instances_tested': 1600, 'conjecture_holds': True}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8bf5d286.py", line 83, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8bf5d286.py", line 83, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~~
KeyError: 'seed'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support. | next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11689
2	preregistration	ollama_remote	glm4:latest	0	0	6020
3	novelty	ollama_remote	glm4:latest	0	0	4921
4	novelty	ollama_remote	glm4:latest	0	0	4968
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15530
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26530
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15157
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14365
9	judge	ollama_remote	glm4:latest	0	0	49638

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148817 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e4f63c2745bc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e4f63c2745bc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e4f63c2745bc.tar.gz` (if generated)